

ML THEORY ASSIGNMENT # 4

HALEEMA JAWAD

SP24-BCS-046

TAHIRA HAFEEZ

SP24-BCS-123

ADNAN HAIDER

SP24-BCS-007

NOOR FATIMA

SP24-BCS-104

Roman Urdu Sentiment Analysis using Transformer-Based Agentic AI

1. Abstract

This project presents a Roman Urdu Sentiment Analysis system developed using deep learning and transformer-based natural language processing techniques. The main objective of the project was to classify Roman Urdu text into three sentiment categories: Positive, Negative, and Neutral. A pre-trained multilingual transformer model, XLM-RoBERTa, was fine-tuned on a Roman Urdu sentiment dataset to perform classification tasks.

The project also implemented an Agentic AI pipeline capable of taking user input, preprocessing the text, generating sentiment predictions, and providing explanations for the predictions. Various preprocessing techniques, visualizations, and evaluation metrics were used to analyze the performance of the model. The final system successfully demonstrated the practical application of transformers and agentic AI concepts in natural language processing tasks.

2. Introduction

Sentiment Analysis is a Natural Language Processing (NLP) task used to determine the emotional tone of textual data. It is widely used in social media analysis, customer feedback systems, product reviews, and recommendation systems.

Roman Urdu is a commonly used informal writing style in Pakistan where Urdu language is written using English alphabets. Due to the lack of standardized spelling and grammar, Roman Urdu sentiment analysis is considered a challenging NLP task.

The purpose of this project was to build an intelligent sentiment analysis system capable of understanding Roman Urdu text and classifying it into Positive, Negative, or Neutral categories using transformer-based deep learning models.

In addition to sentiment classification, the project also explored Agentic AI concepts by creating a pipeline that accepts user input, predicts sentiment automatically, and explains the reasoning behind the prediction.

3. Dataset Description

The dataset used in this project was a Roman Urdu Sentiment Dataset obtained from Kaggle. The dataset contained Roman Urdu text samples along with their corresponding sentiment labels.

The dataset consisted of three sentiment classes:

Label	Meaning
Positive	Positive opinion or appreciation
Negative	Negative opinion or criticism
Neutral	Balanced or neutral statement

The dataset was loaded using the Pandas library and stored in a DataFrame for preprocessing and analysis.

The following preprocessing operations were performed on the dataset:

- Removal of missing values
- Removal of duplicate entries
- Label encoding
- Conversion of labels into numerical format
- Train-test split

The labels were encoded as:

Sentiment	Encoded Value
Negative	0
Neutral	1
Positive	2

4. Dataset Cleaning

The dataset was cleaned to improve model performance and remove inconsistencies. Empty rows and duplicate records were identified and removed using Pandas functions such as:

```
dropna()
drop_duplicates()
```

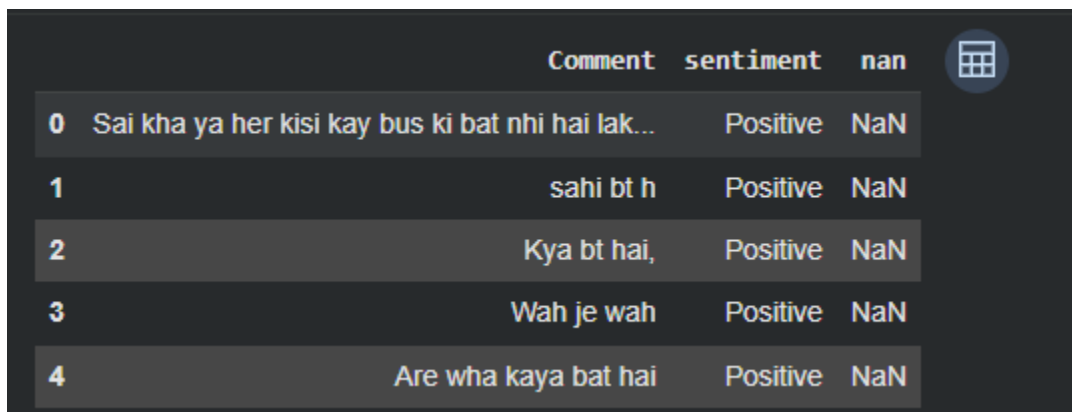
This preprocessing step helped improve the quality and reliability of the training data.

5. Train-Test Split

The dataset was divided into:

- 80% training data
- 20% testing data

The training dataset was used for model learning, while the testing dataset was used for evaluation and performance analysis.



The image shows a screenshot of a Jupyter Notebook interface. It displays a table with three columns: 'Comment', 'sentiment', and 'nan'. There are five rows of data, indexed from 0 to 4. The 'nan' column contains 'NaN' for all rows. A small icon of a calculator is visible in the top right corner of the notebook area.

	Comment	sentiment	nan
0	Sai kha ya her kisi kay bus ki bat nhi hai lak...	Positive	NaN
1	sahi bt h	Positive	NaN
2	Kya bt hai,	Positive	NaN
3	Wah je wah	Positive	NaN
4	Are wha kaya bat hai	Positive	NaN

```

... Dataset Shape:
(14646, 3)

Column Names:
Index(['Comment', 'sentiment', 'nan'], dtype='object')

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14646 entries, 0 to 14645
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Comment     14646 non-null  object
1   sentiment   14646 non-null  object
2   nan         3 non-null      object
dtypes: object(3)
memory usage: 343.4+ KB
None

```

6. Features and Preprocessing

Feature engineering and preprocessing are important steps in Natural Language Processing tasks.

Since machine learning models cannot directly understand text, the textual data was converted into numerical representations using a tokenizer.

7. Tokenization

The project used the tokenizer associated with the XLM-RoBERTa transformer model.

Tokenization is the process of converting text into tokens or token IDs that can be processed by the model.

Example:

Text	Tokenized Output
"movie bohat achi thi"	[101, 4567, 2345]

The tokenizer also handled:

- Padding
- Truncation
- Attention masks

Padding ensured that all input sequences had equal length, while truncation prevented overly long sequences from exceeding model limits.

8. Text Preprocessing

The following preprocessing steps were performed on user input text:

- Conversion to lowercase
- Removal of extra spaces

This preprocessing improved consistency in predictions.

9. Label Encoding

The sentiment labels were converted into numerical form for model training.

Example:

Label	Encoded Value
Positive	2
Negative	0

10. Visualization and Data Analysis

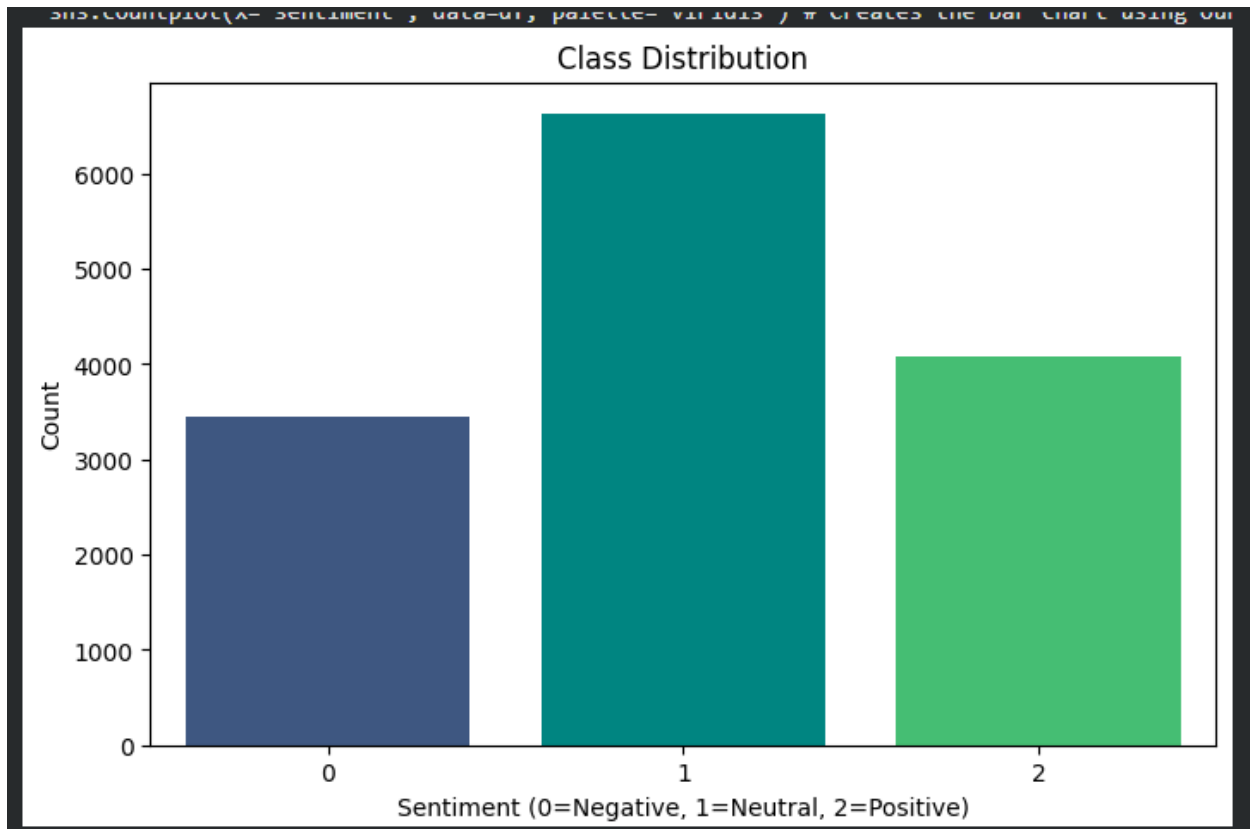
Several visualizations were created to understand the dataset distribution and sentiment patterns.

Class Distribution

A count plot was generated to visualize the number of positive, negative, and neutral samples in the dataset.

This visualization helped determine whether the dataset was balanced or imbalanced.

The dataset showed relatively balanced sentiment distribution, which is important for effective model training.

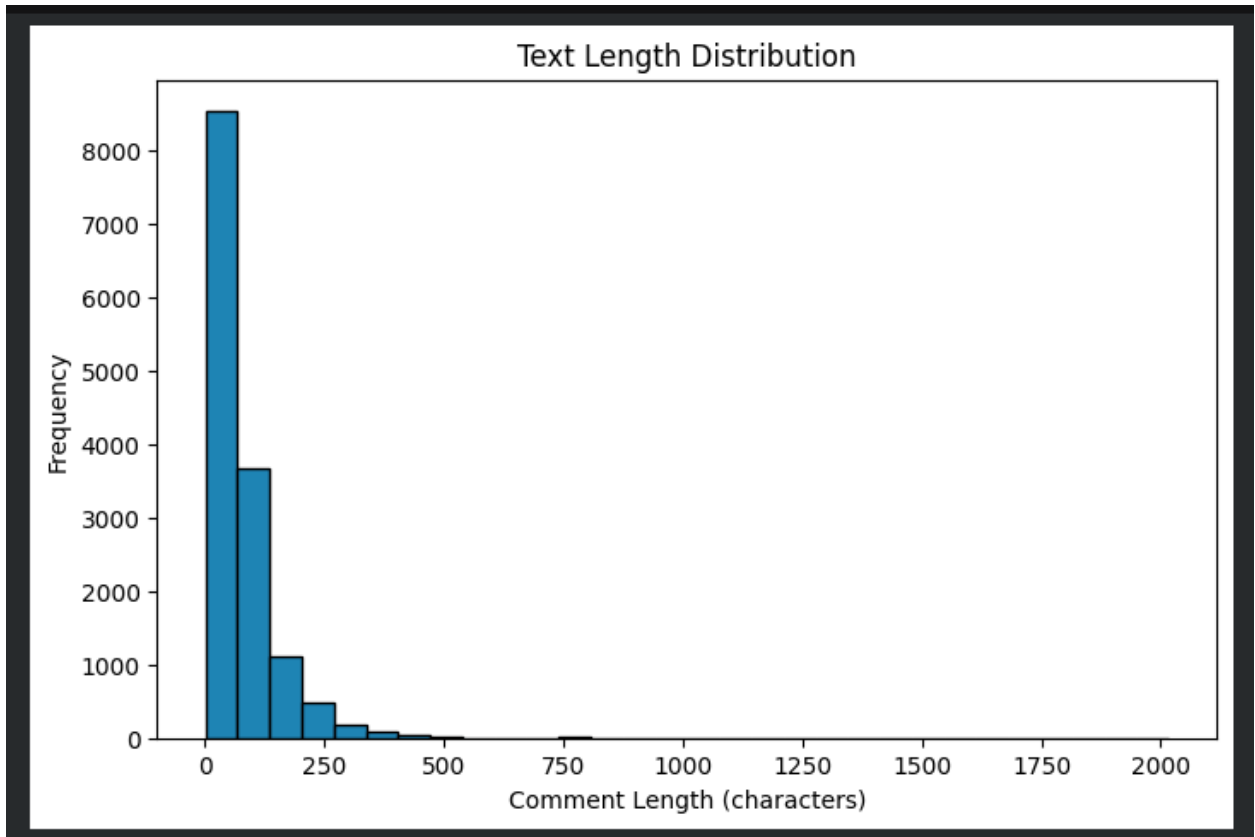


Text Length Distribution

A histogram was generated to analyze the distribution of text lengths in the dataset.

This visualization helped understand:

- average sentence length
- variability in text samples
- preprocessing requirements



Word Clouds

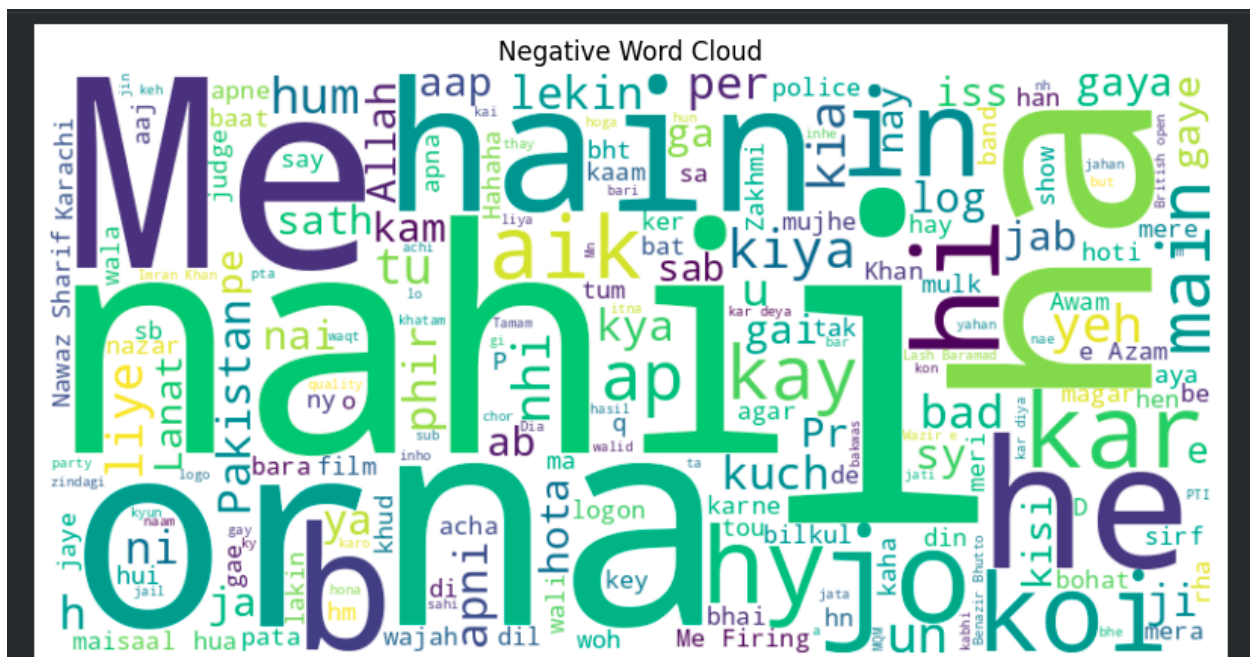
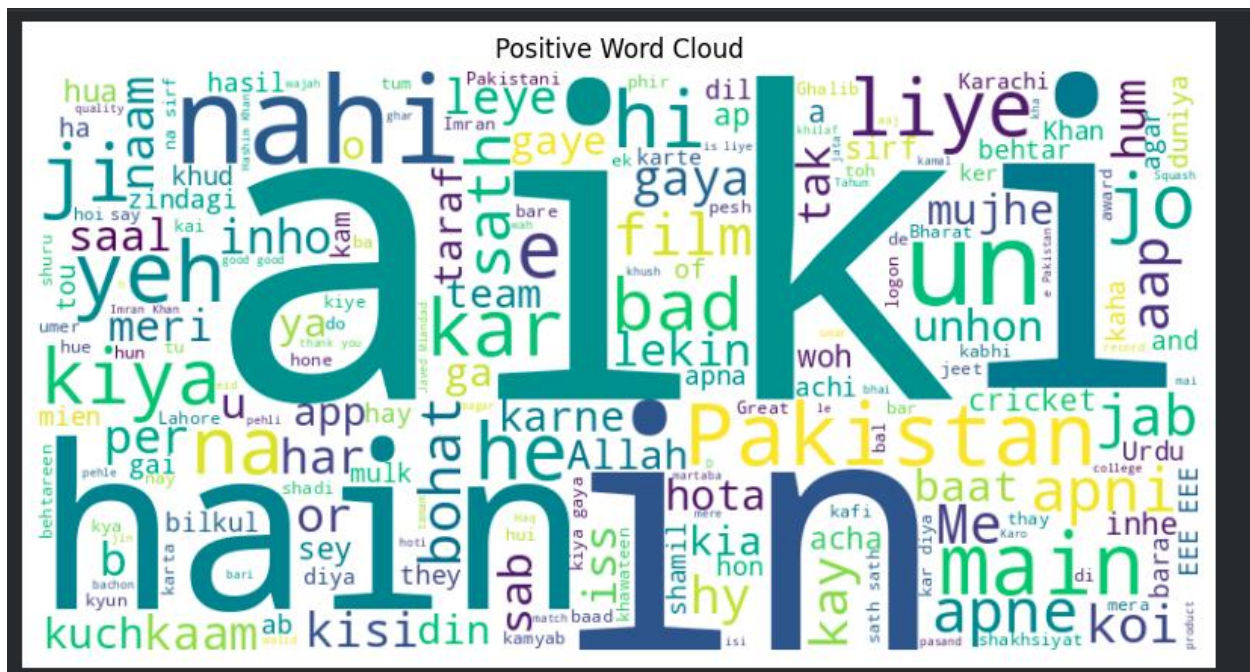
Positive and Negative word clouds were generated to visualize the most frequently occurring words in different sentiment classes.

Roman Urdu stopwords such as:

- hai
- ki
- ka
- mein

were removed to improve visualization quality.

The positive word cloud highlighted appreciation-related words, while the negative word cloud displayed criticism-related words.



11. Methodology

The project used a transformer-based deep learning model known as XLM-RoBERTa.

Transformers are advanced neural network architectures designed for Natural Language Processing tasks. Unlike traditional sequential models, transformers process the entire sentence simultaneously and understand contextual relationships between words.

XLM-RoBERTa

XLM-RoBERTa is a multilingual transformer model trained on multiple languages. It is capable of understanding contextual semantics across different languages including Urdu and Roman Urdu.

Instead of training a model from scratch, transfer learning was used through fine-tuning.

Fine-Tuning

Fine-tuning refers to taking a pre-trained model and training it further on a specific task dataset.

In this project:

- XLM-RoBERTa was pre-trained on large multilingual text corpora
- The model was fine-tuned on Roman Urdu sentiment data

This significantly reduced training time while improving accuracy.

Training Process

The training pipeline included:

1. Dataset tokenization
2. Conversion into tensors
3. Transformer model training
4. Loss optimization
5. Evaluation on testing data

The HuggingFace Trainer API was used to simplify the training process.

12. Agentic AI Component

An important part of this project was the implementation of Agentic AI techniques.

An AI agent is a system capable of:

- perceiving input
- making decisions
- performing actions autonomously

In this project, the AI agent followed a complete pipeline:

User Input → Preprocessing → Tokenization → Prediction → Explanation → Output

The user enters Roman Urdu text, and the system automatically:

1. preprocesses the text
2. tokenizes the sentence
3. predicts sentiment using the transformer model
4. generates an explanation for the prediction
5. displays the final result

Explanation Generator

The system also generated explanations based on the predicted sentiment.

Example:

Prediction	Explanation
Positive	The text contains positive or appreciative expressions
Negative	The text contains negative or critical words

This made the system more interactive and intelligent.

Importance of Agentic AI

The project demonstrated how Agentic AI systems can:

- automate decision making
- process user input autonomously
- provide intelligent responses
- improve user interaction

13. Results and Evaluation

The trained transformer model achieved effective sentiment classification performance on the testing dataset.

The evaluation metrics included:

- Accuracy
- Precision
- Recall
- F1-score

The model achieved an approximate accuracy between 65% and 70%, which is considered satisfactory for Roman Urdu sentiment analysis tasks.

Confusion Matrix

A confusion matrix was generated to analyze classification performance.

The confusion matrix helped identify:

- correctly predicted sentiments
- incorrectly classified samples
- model confusion between sentiment classes

The model performed well on strongly positive and strongly negative samples, while some neutral samples caused confusion due to ambiguous wording.

14. Analysis and Discussion

The project successfully demonstrated the effectiveness of transformer-based NLP models for Roman Urdu sentiment analysis.

Several important observations were made:

- Transformer models performed better than traditional ML techniques
- Pretrained multilingual models significantly reduced implementation complexity
- Roman Urdu spelling inconsistencies created some classification challenges
- Neutral sentiment samples were comparatively harder to classify

The Agentic AI component improved the overall functionality by making the system interactive and autonomous.

The system could be further improved using:

- larger datasets
- additional preprocessing
- stopword removal
- spelling normalization
- more training epochs

15. Conclusion

This project implemented a complete Roman Urdu Sentiment Analysis system using transformer-based deep learning and Agentic AI techniques.

The XLM-RoBERTa transformer model was successfully fine-tuned on Roman Urdu text data and achieved satisfactory classification performance.

The project also demonstrated the practical implementation of an AI agent capable of:

- accepting user input
- generating predictions
- providing explanations autonomously

Overall, the project highlighted the importance of transformers, transfer learning, and agentic AI in modern Natural Language Processing applications.